TaskAces — Project Handoff Document

App Name: TaskAces
Version: 1.0.5
Platform: iOS & Android (React Native / Expo)
Repository: https://github.com/qualaces/mobileAssistantApp.git
Date: April 2026

Part 1 — Technical Documentation

1.1 Architecture Diagrams

System Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                          TaskAces App                            │
│                     (React Native / Expo)                        │
│                                                                  │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐    │
│   │   app/       │   │   lib/       │   │   contexts/      │    │
│   │  (Screens)   │   │ (Utilities)  │   │  (State/Theme)   │    │
│   └──────────────┘   └──────────────┘   └──────────────────┘    │
│          │                  │                    │              │
│          └──────────────────┴────────────────────┘              │
│                             │                                    │
│                ┌────────────┴──────────────┐                    │
│                │    Local Storage          │                    │
│                │   (AsyncStorage)          │                    │
│                └───────────────────────────┘                    │
└──────────────────────────────┬───────────────────────────────────┘
                               │ (optional cloud sync)
              ┌────────────────┴────────────────┐
              │                │                │
        ┌─────▼──────┐   ┌─────▼──────┐   ┌─────▼──────┐
        │  Supabase  │   │Google OAuth│   │  Expo Push │
        │ (Postgres) │   │  Sign-In   │   │Notification│
        │ + Storage  │   │            │   │  Service   │
        └────────────┘   └────────────┘   └────────────┘
```

Navigation Architecture

```
Stack Navigator (app/_layout.tsx)
│
├── / → redirects to /(tabs)/lists
│
├── (tabs)/   (Bottom Tab Navigator)
│   ├── lists        → Task List Screen
│   └── settings     → Settings Screen
│
└── task/[id]        → Task Detail/Edit Screen
```

```

### Data Flow

```
User Action
    │
    ▼
Screen Component (app/(tabs)/lists.tsx or app/task/[id].tsx)
    │
    ├── Reads from AsyncStorage (@tasks_storage)
    ├── Writes to AsyncStorage (@tasks_storage)
    │
    └── [Optional] Supabase sync (lib/supabase.ts)
            │
            └── PostgreSQL (tasks, task_photos, lists tables)
```

### Database Schema (Supabase)

```
tasks
  ├── id            UUID (PK)
  ├── user_id       UUID (FK → auth.users)
  ├── task_number   INTEGER
  ├── title         TEXT
  ├── description   TEXT
  ├── priority      TEXT ('P1' | 'P2' | 'P3')
  ├── due_date      TIMESTAMP
  ├── completed     BOOLEAN
  ├── list_name     TEXT
  ├── status_emoji  TEXT
  ├── created_at    TIMESTAMP
  └── updated_at    TIMESTAMP

task_photos
  ├── id            UUID (PK)
  ├── task_id       UUID (FK → tasks.id)
  ├── photo_uri     TEXT
  └── created_at    TIMESTAMP

lists
  ├── id            UUID (PK)
  ├── user_id       UUID (FK → auth.users)
  ├── name          TEXT
  ├── color         TEXT
  └── created_at    TIMESTAMP
```

1.2 File Navigation

```
project_2/
```
```

```
├── app/                          # All screens (Expo Router)
│   ├── _layout.tsx               # Root Stack layout, theme/gesture setup
│   ├── index.tsx                 # Entry point — redirects to /lists
│   ├── +not-found.tsx            # 404 fallback screen
│   ├── (tabs)/
│   │   ├── _layout.tsx           # Bottom tab bar (Lists + Settings)
│   │   ├── lists.tsx             # Primary screen — task management (900+ lines)
│   │   └── settings.tsx          # Dark mode toggle & Google Sign-In
│   ├── task/
│   │   └── [id].tsx              # Dynamic task detail/edit screen (745 lines)
│   └── temp/
│       └── _today.tsx            # WIP — today view (not yet released)
│
├── lib/                          # Stateless utility modules
│   ├── supabase.ts               # Supabase client initialization
│   ├── googleAuth.ts             # Google Sign-In helpers
│   ├── notifications.ts          # Expo push notification helpers
│   ├── nlp.ts                    # NLP parser for task input
│   └── localAi.ts                # Local task generation (no API call)
│
├── contexts/
│   └── ThemeContext.tsx           # Light/dark theme provider + hook
│
├── hooks/
│   └── useFrameworkReady.ts      # Expo framework initialization hook
│
├── supabase/
│   ├── migrations/               # SQL schema migrations
│   └── functions/                # Supabase Edge Functions
│
├── assets/                       # Images, icons, splash screens
├── android/                      # Native Android project files
├── ios/                          # Native iOS project files
│
├── app.json                      # Expo app config (name, bundle IDs, version)
├── eas.json                      # EAS build profiles (dev/preview/production)
├── babel.config.js               # Babel config (expo preset + reanimated)
├── tsconfig.json                 # TypeScript config (strict, path alias @/*)
├── package.json                  # Dependencies, scripts, metadata
└── .env                          # Environment variables (not committed to git)
```

Key path alias: `@/*` maps to the project root, configured in tsconfig.json.

1.3 Installation Process

Prerequisites

Tool	Version	Notes
Node.js	18 or 20 LTS	Required
npm	Bundled with Node	Or yarn/bun

Expo CLI	Latest	`npm install -g expo-cli`
EAS CLI	>= 16.19.3	`npm install -g eas-cli`
Xcode	15+	iOS builds only (macOS)
Android Studio	Latest	Android builds only
Expo Go app	Latest	For quick device preview

Steps

```bash
# 1. Clone the repository
git clone https://github.com/qualaces/mobileAssistantApp.git
cd mobileAssistantApp

# 2. Install dependencies
npm install

# 3. Set up environment variables
cp .env.example .env   # if .env.example exists, otherwise create .env manually
# Fill in the values listed in the Environment Variables section below

# 4. Start the development server
npm run dev

# 5. Run on a device or simulator
npm run ios       # iOS Simulator (macOS only)
npm run android   # Android emulator or connected device
```

Environment Variables

Create a `.env` file in the project root with the following keys:

```env
# Google OAuth Client IDs (from Google Cloud Console)
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=<your-web-client-id>
EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID=<your-ios-client-id>
EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID=<your-android-client-id>

# Supabase (from your Supabase project dashboard)
EXPO_PUBLIC_SUPABASE_URL=<your-supabase-url>
EXPO_PUBLIC_SUPABASE_ANON_KEY=<your-supabase-anon-key>
```

EAS (Cloud) Builds

```bash
# Authenticate with Expo
eas login

# Build for development (device testing)
eas build --profile development --platform ios
eas build --profile development --platform android

# Build for production (App Store / Play Store)
eas build --profile production --platform all
```

```

**EAS Project ID:** `da173da0-3ef8-4941-89b2-94641ee4caf5`
**App Store Connect App ID:** `6755730110`
**Apple ID:** `saulmacias5142004@gmail.com`

---

## 1.4 Licenses

The project does not include an explicit `LICENSE` file in the repository. All first-party code is private (`"private": true` in package.json).

Third-party libraries are governed by their respective open-source licenses — primarily **MIT**. Notable exceptions:

| Library | License |
|---|---|
| React Native | MIT |
| Expo SDK | MIT |
| Supabase JS | MIT |
| NativeWind | MIT |
| React Native Reanimated | MIT |
| React Native Draggable FlatList | MIT |
| Lucide React Native | ISC |
| AsyncStorage | MIT |

To generate a full license report run:

```bash
npx license-checker --summary
```

1.5 Libraries

Runtime Dependencies

Library	Version	Source	Purpose
react	19.1.0	npmjs.com	UI framework
react-native	0.81.5	npmjs.com	Mobile runtime
expo	~54.0.33	npmjs.com	Managed workflow, dev tooling
expo-router	~6.0.23	npmjs.com	File-based navigation
@react-navigation/bottom-tabs	^7.3.10	npmjs.com	Bottom tab navigator
@react-navigation/native	^7.1.6	npmjs.com	Navigation container
nativewind	^4.1.23	npmjs.com	Tailwind CSS for React Native
react-native-reanimated	~4.1.1	npmjs.com	Gesture animations
react-native-gesture-handler	~2.23.1	npmjs.com	Touch gesture support
react-native-draggable-flatlist	^4.0.1	npmjs.com	Drag-to-reorder lists
@supabase/supabase-js	^2.58.0	npmjs.com	Backend client (auth, DB, storage)
@react-native-async-storage/async-storage	^2.2.0	npmjs.com	Local persistence
expo-notifications	~0.31.2	npmjs.com	Push notifications
```

@react-native-google-signin/google-signin	^15.0.0	npmjs.com	Google OAuth sign-in
expo-auth-session	~6.1.4	npmjs.com	OAuth session handling
expo-image-picker	~16.1.4	npmjs.com	Camera roll access
expo-camera	~16.1.6	npmjs.com	In-app camera
expo-file-system	~18.1.1	npmjs.com	File I/O for photos
lucide-react-native	^0.475.0	npmjs.com	Icon library
@expo/vector-icons	^14.0.4	npmjs.com	Extended icon set
react-native-safe-area-context	4.16.1	npmjs.com	Safe area insets
react-native-screens	~4.11.1	npmjs.com	Native screen containers
react-native-url-polyfill	^2.0.0	npmjs.com	URL API polyfill (Supabase req.)

Dev Dependencies

Library	Version	Purpose
typescript	~5.9.2	Static typing
@babel/core	^7.25.2	Transpilation
babel-preset-expo	~13.0.0	Expo Babel preset
expo-constants	~17.1.6	Build constants access
expo-linking	~7.1.4	Deep linking
expo-splash-screen	~0.30.8	Splash screen control
expo-status-bar	~2.2.3	Status bar theming
expo-web-browser	~14.1.0	OAuth web browser

1.6 APIs and SDKs

Supabase

Property	Value
SDK	`@supabase/supabase-js` v2.58.0
Source	https://supabase.com / npmjs.com
Auth	Row Level Security (RLS) per user
Config file	lib/supabase.ts
Services used	Auth, PostgreSQL Database, Storage

Supabase is initialized with `AsyncStorage` as the auth session storage so sessions persist across app restarts.

Google Sign-In

Property	Value
SDK	`@react-native-google-signin/google-signin` v15.0.0
Source	npmjs.com / Google Cloud Console
Config file	lib/googleAuth.ts
Client IDs	Set via `.env` (Web, iOS, Android)

Expo Notifications

Property	Value

SDK	`expo-notifications` ~0.31.2
Source	npmjs.com / Expo
Config file	lib/notifications.ts
Behavior	Schedules local notifications at 12:00 PM for task due dates

Expo Application Services (EAS)

Property	Value
CLI	`eas-cli` >= 16.19.3
Source	https://expo.dev/eas
Config file	eas.json
Used for	Cloud builds, OTA updates, App Store submission

Part 2 — User Guide

2.1 How the System Works

TaskAces is a mobile task management application built for iOS and Android. It allows users to create, organize, prioritize, and track tasks across multiple lists. The app is **local-first**: all task data is saved on the device using AsyncStorage, meaning the app works fully offline without any account. An optional Google Sign-In connects to a Supabase cloud backend for future cross-device sync.

Tasks are organized by **priority tier** (P1, P2, P3) and by **time-based lists** (Inbox, Today, This Week, This Month). Users can drag tasks between priority levels, attach photos, add status emojis, and receive push notification reminders for due dates. A dark mode is available and persists between sessions.

2.2 Goals and Functionality

EPIC 1 — Task Management

As a user, I want to create tasks so that I can track what I need to do.
- Create a task with a title and optional due date
- Tasks are assigned to one of three priority levels: P1 (High), P2 (Medium), P3 (Low)
- Tasks can be assigned to a named list (e.g., "Work", "Personal")

As a user, I want to organize tasks by priority so that I focus on what matters most.
- View tasks grouped under P1, P2, P3 headers
- Drag tasks between priority groups to re-rank them
- Priority groups are collapsible

As a user, I want to mark tasks complete so that I can track my progress.
- Tap a checkbox to mark a task done
- Completed tasks move to a collapsible "Completed" section at the bottom

As a user, I want to delete tasks so that I can remove things I no longer need.
- Swipe left on any task to reveal a delete action
- Delete from within the task detail screen

EPIC 2 — Task Views & Filtering

As a user, I want to filter tasks by time frame so that I focus on what's relevant now.
- **Inbox** — all tasks
- **Today** — tasks due today
- **This Week** — tasks due within the current week
- **This Month** — tasks due within the current month

As a user, I want to sort tasks so that I can see them in my preferred order.
- Sort options: Created date, Priority, Due date, Alphabetical

EPIC 3 — Task Detail & Enrichment

As a user, I want to view and edit task details so that I can update tasks as things change.
- Edit title, priority, and due date from the detail screen
- Access via tapping a task row

As a user, I want to attach photos to a task so that I can store visual context.
- Take a photo with the camera or select from the device gallery
- Photos appear in a scrollable list on the task detail screen
- Pinch-to-zoom photo viewer

As a user, I want to add a status emoji to a task so that I can communicate its state at a glance.
- Quick-pick from 4 preset emojis
- Or enter any custom emoji

EPIC 4 — Notifications

As a user, I want to receive reminders for upcoming tasks so that I don't miss due dates.
- When a due date is set, a local notification is scheduled for 12:00 PM on that day
- Notifications are rescheduled when due dates change

EPIC 5 — Appearance

As a user, I want a dark mode so that I can use the app comfortably at night.
- Toggle dark/light mode from the Settings screen
- Preference persists across app restarts

EPIC 6 — Account (Optional)

As a user, I want to sign in with Google so that my tasks can sync across devices in the future.
- Google Sign-In from the Settings screen
- App works fully without an account (local storage fallback)

2.3 Inputs and Outputs

Inputs

Input	Where	Description
Task title	New task modal / Task detail	Text field, required
Priority	New task modal / Task detail	P1, P2, or P3 selector
Due date	New task modal / Task detail	Date picker
List name	New task modal	Text field (defaults to "Inbox")
Status emoji	Task detail	Quick emoji picker or custom
Photos	Task detail	Camera or gallery picker
Sort preference	Lists screen toolbar	Picker (date/priority/due/alpha)
Filter (list view)	Lists screen tab bar	Inbox / Today / This Week / This Month
Dark mode toggle	Settings screen	On/off switch
Google account	Settings screen	OAuth sign-in flow

Outputs

Output	Where	Description
Task list	Lists screen	Priority-grouped, filtered task rows
Task count badges	Priority section headers	Count of tasks per priority
Completed section	Lists screen (bottom)	Collapsed list of finished tasks
Task detail view	Task detail screen	All task fields + photos
Push notification	Device notification tray	Reminder at 12:00 PM on due date
User profile	Settings screen	Avatar + name when signed into Google
Theme	App-wide	Light or dark color scheme
Persisted data	AsyncStorage	Survives app restarts and reboots

---

2.4 Services and Features

Local Storage (AsyncStorage)

All tasks are stored locally under the key `@tasks_storage` as a JSON array. This is the primary datastore. No internet connection is required for core functionality. Theme preference is stored under a separate key and persists independently.

Cloud Backend (Supabase — Optional)

Supabase provides a PostgreSQL database, authentication, and file storage for future multi-device sync. Row Level Security (RLS) ensures each user can only access their own data. The integration is initialized in lib/supabase.ts but the app falls back gracefully to local storage when the user is not signed in.

Google Authentication

Handled by `@react-native-google-signin/google-signin`. Supports sign-in, sign-out, and fetching the current user profile. Authentication state is surfaced in the Settings screen and is not required to use the app. See lib/googleAuth.ts.

Push Notifications

Implemented via `expo-notifications`. When a task is created or updated with a due date, a local notification is scheduled for 12:00 PM on that date. Notifications are

cancelled and re-created when due dates change. Android uses a high-importance notification channel. See lib/notifications.ts.

NLP Task Parsing

A lightweight, no-API natural language parser lives in [lib/nlp.ts](lib/nlp.ts). It can extract:
- **Priority** from words like "urgent", "high", "P1"
- **Due dates** from phrases like "today", "tomorrow", "next week"
- **Times** from patterns like "3pm", "10:30 AM"

This module is implemented but not yet wired to the main task creation UI.

Theme System

A React Context (contexts/ThemeContext.tsx) provides light/dark theming to the entire app. Colors are defined as a static palette (primary blue `#2196F3`, secondary green `#4CAF50`, and surface colors per theme). The user's choice is read from AsyncStorage on launch and defaults to the system color scheme.

Drag-to-Reorder

Powered by `react-native-draggable-flatlist`, tasks within each priority group can be reordered by long-pressing and dragging. Order is persisted to AsyncStorage.

Swipe-to-Delete

Implemented with `react-native-gesture-handler`. Swiping a task row to the left reveals a red delete action. The deletion is immediate and updates AsyncStorage.

Photo Attachments

Users can attach photos to tasks from the camera (`expo-camera`) or the device gallery (`expo-image-picker`). Photos are stored as local file URIs on the device and displayed in a scrollable row on the task detail screen with a pinch-to-zoom viewer.

TaskAces — Best Practices Document

App Name: TaskAces
Version: 1.0.5
Platform: iOS & Android (React Native / Expo)
Date: April 2026

---

1. Code Updates and Maintenance

1.1 Follow a Consistent Branching Strategy

Use **Git Flow** or a simplified variant:

Branch	Purpose
`main`	Stable, production-ready code only
`develop`	Integration branch for completed features

`feature/<name>`	Individual feature development
`fix/<name>`	Bug fixes
`release/<version>`	Pre-release stabilization

Never commit directly to `main`. All changes should arrive via a Pull Request with at least one reviewer approval. The current branch naming convention (`TaskAces_1.0.0`) should be reserved for release branches, not general development work.

1.2 Write Meaningful Commit Messages

Follow the **Conventional Commits** specification:

```
<type>(<scope>): <short summary>

feat(lists): add swipe-to-delete on task rows
fix(notifications): reschedule notification when due date changes
chore(deps): upgrade expo-notifications to 0.32.0
refactor(theme): extract color tokens to ThemeContext
```

Types: `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`.
This makes changelogs and release notes easy to generate automatically.

1.3 Keep Components Focused and Small

- Each screen file (`lists.tsx`, `task/[id].tsx`) should own its UI and local state only. Business logic (data fetching, storage reads/writes, notification scheduling) belongs in dedicated utility modules under `lib/`.
- `lists.tsx` is currently ~900 lines. Consider extracting sub-components (e.g., `TaskRow`, `PrioritySection`, `AddTaskModal`) into a `components/` directory to improve readability and testability.

1.4 TypeScript Strict Mode

The project already has `strict: true` in tsconfig.json. Maintain this — do not disable or suppress TypeScript errors with `// @ts-ignore` or `any` casts. Instead, write proper type definitions. This catches bugs at compile time before they reach users.

1.5 Environment Variables

- All secrets and environment-specific values must live in `.env` and never be hard-coded in source files.
- `.env` must remain in `.gitignore` — it is never committed.
- Prefix all client-side env vars with `EXPO_PUBLIC_` so Expo bundles them correctly.
- Maintain a `.env.example` file in the repository documenting every required key (with placeholder values) so new developers know what to configure.

1.6 Code Reviews

- All Pull Requests into `develop` or `main` require a review before merge.
- Review checklist: no hard-coded secrets, TypeScript errors resolved, no unused imports, feature tested on both iOS and Android.

- Keep PRs small and focused — one feature or fix per PR makes review faster and rollbacks cleaner.

1.7 Testing

- Write unit tests for pure utility functions in `lib/` (e.g., the NLP parser in `lib/nlp.ts`, date helpers).
- Use Expo's testing setup with **Jest** and `@testing-library/react-native` for component tests.
- Before any release build, manually verify the golden path on a physical device: create a task, set a due date, receive a notification, attach a photo, toggle dark mode.

1.8 Linting and Formatting

Run linting before every commit:

```bash
npm run lint        # ESLint
npm run typecheck   # TypeScript
```

Consider adding a pre-commit hook (via `husky` + `lint-staged`) to enforce this automatically so broken code can never be committed.

---

2. APIs, SDKs, and Libraries — Updates and Maintenance

2.1 Audit Dependencies Regularly

Run a dependency audit at least once per month:

```bash
npm audit                  # check for known vulnerabilities
npm outdated               # list packages with newer versions
```

Address any **high** or **critical** severity vulnerabilities immediately. Medium severity should be resolved within the next sprint.

2.2 Update Strategy

Do not update everything at once. Follow this priority order:

1. **Security patches** — update immediately regardless of other work.
2. **Expo SDK** — Expo releases a new SDK roughly every quarter. Upgrade within 6 weeks of each release. Follow the official [Expo upgrade guide](https://docs.expo.dev/workflow/upgrading-expo-sdk-walkthrough/) — it lists breaking changes and required dependency bumps. The current SDK 54 was chosen specifically because SDK 55 introduced a worklets incompatibility with Reanimated.
3. **React Native** — tied to the Expo SDK version; upgrade together.
4. **Supabase JS** — follow their changelog for breaking changes to auth or query APIs before upgrading.
5. **Other libraries** — batch minor/patch updates monthly.

2.3 Lock File Discipline

Always commit `package-lock.json`. This ensures every developer and every CI build installs the exact same dependency tree. Never delete and regenerate the lock file unnecessarily.

2.4 Test After Every Dependency Update

After any library upgrade:
1. Run `npm install`
2. Run `npm run typecheck`
3. Start the dev server and test the affected features on a device
4. Run a clean EAS development build before marking the upgrade complete

2.5 Watch for Expo SDK Deprecations

The Expo team deprecates APIs with advance notice. Subscribe to the [Expo blog](https://expo.dev/blog) and review the SDK changelog on each release. Pay particular attention to:
- `expo-notifications` API changes (notification scheduling format has changed across SDK versions)
- `expo-auth-session` OAuth flow changes
- Expo Router navigation API updates

2.6 Google Sign-In SDK

`@react-native-google-signin/google-signin` requires the Google Cloud Console client IDs to remain active. If client IDs are rotated or OAuth scopes change, update `.env` and re-test the sign-in flow on both platforms before releasing.

2.7 Supabase Client SDK

Supabase periodically releases breaking changes to `@supabase/supabase-js`. Before upgrading across major versions:
- Read the Supabase migration guide
- Test all auth flows (sign-in, session persistence, sign-out)
- Verify RLS policies still apply correctly after any schema changes

---

3. Repository and Versioning Management

3.1 Semantic Versioning

TaskAces uses semantic versioning (`MAJOR.MINOR.PATCH`):

Segment	Increment when...
MAJOR	Breaking change visible to users (complete redesign, data migration required)
MINOR	New feature added in a backwards-compatible way
PATCH	Bug fix, performance improvement, or small UI tweak

Current version `1.0.5` is in app.json and package.json. Both files must be updated together on every release.

3.2 Build Numbers

- **iOS** build numbers and **Android** version codes are managed by EAS (`"autoIncrement": true` in eas.json).
- Never manually edit build numbers in `ios/` or `android/` native files — let EAS handle this to avoid conflicts.
- The `"versionSource": "remote"` setting in `eas.json` means EAS tracks the authoritative version, not local files.

3.3 Tagging Releases

Every production release must be tagged in Git:

```bash
git tag -a v1.0.5 -m "Release 1.0.5 — dark mode persistence fix"
git push origin v1.0.5
```

This creates a permanent, traceable snapshot of exactly what was shipped to the App Store and Play Store.

3.4 Release Notes

Maintain a `CHANGELOG.md` file in the repository root. Update it with every release using the format:

```markdown
## [1.0.5] — 2026-04-05
### Fixed
- Dark mode preference now persists correctly after app restart
### Changed
- Upgraded expo-notifications to 0.31.2
```

Use commit history and Conventional Commits to generate this semi-automatically.

3.5 Branch Protection Rules

In GitHub, configure the following on the `main` branch:
- Require pull request reviews before merging (minimum 1 approver)
- Require status checks to pass (linting, type checks)
- Disallow force pushes
- Disallow branch deletion

3.6 Repository Hygiene

- Archive or delete stale branches after merging.
- The `temp/` directory in `app/temp/` contains WIP features (`_today.tsx`) — move in-progress work to feature branches instead of committing unfinished files to `main`.
- Never commit `.env`, `node_modules/`, build artifacts (`android/app/build/`, `ios/build/`), or EAS credentials.

---

4. Licensing Management

4.1 Establish a Project License

The repository currently has no `LICENSE` file. This means legally, all rights are reserved and no one else can use, modify, or distribute the code. Decide on and add a license:

License	Best for
MIT	Permissive — others can use the code freely
Apache 2.0	Permissive with patent protection
GPL v3	Copyleft — derivative works must remain open source
Proprietary	Closed-source commercial product

For a capstone or commercial mobile app, **MIT** (open) or a **proprietary/all-rights-reserved** statement is most common. Add the chosen license as `LICENSE` in the repository root and reference it in `README.md`.

4.2 Third-Party License Compliance

All current dependencies are MIT-licensed, which is highly permissive — no special action is required beyond attribution. However:

- Run `npx license-checker --summary` regularly to detect if any new dependency introduces a restrictive license (GPL, AGPL, CC-BY-NC) that could conflict with commercial distribution.
- If a copyleft dependency is added, legal review is required before shipping to the App Store or Play Store.

4.3 App Store and Play Store Requirements

Both Apple and Google require that apps disclose third-party software. When submitting:
- Apple: no explicit EULA is required for MIT libraries, but include a privacy policy URL.
- Google Play: similarly requires a privacy policy in the store listing.

4.4 API and SDK License Terms

Service	Key Term to Watch
Google Sign-In	Google API Terms of Service — do not use Sign-In for restricted scopes without review
Supabase	Free tier has row/bandwidth limits; commercial use is permitted
Expo / EAS	Free tier has build limits; EAS Production plan required for high-volume builds
Expo Notifications	No special license; Expo Push Service is free with rate limits

Review these terms annually or when upgrading to a paid plan.

---

5. Hosting and Domain Services

5.1 Supabase Project Management

TaskAces uses Supabase for its cloud backend.

- **Do not use the free tier for production.** The Supabase free tier pauses projects after 1 week of inactivity and has limited storage. Upgrade to the **Pro plan** before a public release.
- Enable **Point-in-Time Recovery (PITR)** on the Pro plan so the database can be restored to any moment within the retention window.
- Rotate the Supabase `anon` and `service_role` keys annually or immediately if any key is accidentally exposed in a commit.
- Set up **database backups** — Supabase Pro includes daily backups automatically.
- Use the Supabase dashboard to monitor database size, auth users, and storage usage monthly.

5.2 Supabase Edge Functions

Edge Functions in `supabase/functions/` are deployed via the Supabase CLI:

```bash
supabase functions deploy <function-name>
```

- Keep Edge Functions small and single-purpose.
- Store any secrets they need (API keys) in Supabase Secrets, not in function source code.
- Test functions locally with `supabase functions serve` before deploying.

5.3 Expo Application Services (EAS)

EAS hosts OTA update bundles and build artifacts.

- Enable **OTA (Over-the-Air) updates** via `expo-updates` for minor bug fixes — this allows pushing JS-layer fixes without App Store/Play Store review.
- Reserve native binary builds for changes that touch native code, new permissions, or SDK upgrades.
- Use EAS **channels** (`production`, `preview`) to target updates to the right user group.
- Store EAS credentials securely — do not commit `credentials.json` or Apple/Google key files.

5.4 No Custom Domain Required

TaskAces is a mobile app with no web frontend. Supabase provides its own API URL (e.g., `https://<project-ref>.supabase.co`). If a marketing site or support page is added in the future, use a registrar such as Namecheap or Cloudflare Registrar, and point DNS to the hosting provider (Vercel, Netlify, etc.).

---

6. Software Distribution Platforms

6.1 Apple App Store

Current config:
- Bundle ID: `com.zearot.boltexponativewind`
- App Store Connect App ID: `6755730110`
- Apple ID: `saulmacias5142004@gmail.com`

Best practices:

- **Two-factor authentication:** Ensure the Apple Developer account has 2FA enabled. A compromised account can push malicious updates to all users.
- **App Store Connect API Key:** Use an API key (not a password) for EAS to submit builds — this avoids storing your Apple ID password anywhere.
- **Certificates and provisioning profiles:** Managed automatically by EAS (`"credentialsSource": "remote"` pattern). Do not manually create or revoke certificates through Xcode unless absolutely necessary — this can break EAS builds.
- **Privacy policy:** Apple requires a privacy policy URL in the store listing. Host it on a stable URL (GitHub Pages, Notion, etc.) and never let it go offline.
- **Review guidelines:** Before submitting, review [App Store Review Guidelines](https://developer.apple.com/app-store/review/guidelines/). Notifications, photo access, and camera usage require a purpose string in `app.json` explaining why the app needs each permission.
- **TestFlight:** Always release to TestFlight internal testers first and test for at least 48 hours before submitting for App Store review.

6.2 Google Play Store

Current config:
- Package name: `com.brunnhilde14.boltexponativewind`

Best practices:

- **Service account:** Create a Google Play service account in Google Cloud Console and upload the JSON key to EAS for automated submissions. Never use your personal Google account credentials.
- **Play App Signing:** Enroll in Play App Signing (Google holds the signing key). This protects against lost keystores and allows recovery if a keystore is accidentally deleted.
- **Internal testing track:** Use the Internal Testing track (up to 100 testers) before promoting to Production. Promotion is instant — no review delay.
- **Target API level:** Google enforces a minimum target SDK level. Keep `targetSdkVersion` current (React Native and Expo handle this, but verify on each SDK upgrade).
- **Data safety form:** Complete the Data Safety section in Play Console, disclosing what data the app collects (task data stored locally, Google account info if signed in, photos).

6.3 OTA Updates (Expo EAS Update)

For JS-only changes (bug fixes, copy changes, minor UI tweaks):

```bash
eas update --branch production --message "Fix notification scheduling bug"
```

- OTA updates bypass App Store/Play Store review and reach users within minutes.
- OTA updates cannot change native modules, permissions, or app icons — those require a full binary release.
- Use OTA updates for urgency; use binary releases for anything touching native code.

---

7. Database Services

7.1 Supabase Schema Migrations

Database schema changes must always be managed through migration files, never by editing tables directly in the Supabase dashboard.

```bash
# Create a new migration
supabase migration new add_task_notes_column

# Apply locally for testing
supabase db reset

# Push to production
supabase db push
```

Migration files live in `supabase/migrations/` and are committed to the repository so the schema history is version-controlled alongside the application code.

7.2 Row Level Security (RLS)

RLS is already enabled on all tables (`tasks`, `task_photos`, `lists`). This is critical — it ensures that a signed-in user can only read and write their own rows, enforced at the database level regardless of application logic.

Rules to follow:
- **Never disable RLS** on any table that holds user data.
- Every new table must have RLS enabled and a corresponding policy before it is used by the app.
- Test RLS policies by attempting to access another user's data with a different auth token — the query should return zero rows.

7.3 Backups and Recovery

Tier	Backup approach
Development	None required — use `supabase db reset` to rebuild from migrations
Staging	Weekly manual export via `supabase db dump`
Production	Enable Supabase Pro daily backups; enable PITR for the ability to restore to any point in time

Store exported backups in a separate location from the primary database (e.g., an encrypted S3 bucket or Google Drive folder with restricted access).

7.4 Local Storage (AsyncStorage)

AsyncStorage is the primary datastore for tasks. It is stored unencrypted on the device's filesystem.

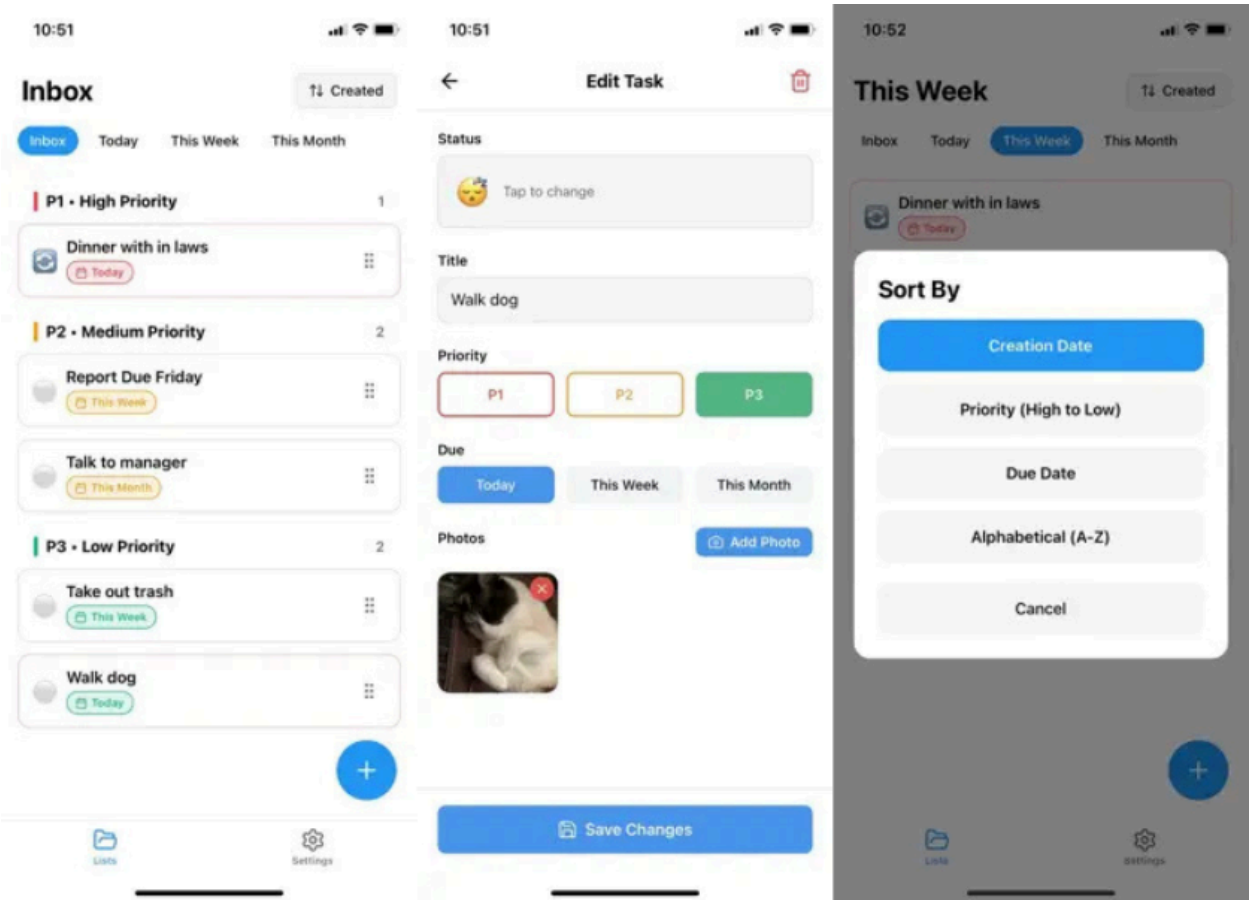
- **Do not store sensitive data** (passwords, tokens, PII beyond task titles) in AsyncStorage without encryption.
- Auth tokens from Supabase and Google Sign-In are already stored via the respective SDKs' secure storage — do not duplicate them into AsyncStorage manually.
- The `@tasks_storage` key stores the full task array as a JSON string. As data grows, consider paginating or indexing reads rather than loading the entire array on every screen mount.
- If cloud sync is added in the future, implement a **conflict resolution strategy** (e.g., last-write-wins with `updated_at` timestamps) before merging local and remote data.

7.5 Performance

- Keep Supabase queries filtered and scoped — always include a `user_id` filter in every query so the database uses the index instead of scanning all rows.
- Avoid `select(*)` in production queries — select only the columns the app needs.
- Add a database index on `tasks.user_id` and `tasks.due_date` to support the filtering views (Today, This Week, This Month) efficiently.
- Monitor query performance in the Supabase dashboard under **Database > Query Performance**.

7.6 Data Retention and Privacy

- Define a data retention policy before public launch. For example: deleted tasks are soft-deleted (a `deleted_at` column) for 30 days, then purged.
- If a user requests account deletion (required under GDPR / CCPA), implement a cascade delete that removes all rows tied to their `user_id` across all tables.
- Document what data is collected, where it is stored, and for how long in the app's privacy policy.


10:51
Inbox
Created
Inbox
Today
This Week
This Month
P1 • High Priority
1
Dinner with in laws
Today
P2 • Medium Priority
2
Report Due Friday
This Week
Talk to manager
This Month
P3 • Low Priority
2
Take out trash
This Week
Walk dog
Today
Lists
Settings
10:51
Edit Task
Status
Tap to change
Title
Walk dog
Priority
P1
P2
P3
Due
Today
This Week
This Month
Photos
Add Photo
Save Changes
10:52
This Week
Created
Inbox
Today
This Week
This Month
Dinner with in laws
Sort By
Creation Date
Priority (High to Low)
Due Date
Alphabetical (A-Z)
Cancel